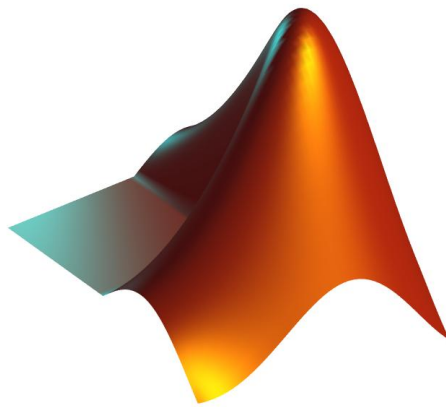


Iniciação ao *MATLAB*

Versão resumida



Universidade do Minho
Escola de Ciências
Departamento de Matemática

maria irene falcão

Conteúdo

1	Iniciação ao MATLAB	1
1.	Introdução	1
	O sistema MATLAB	1
	Iniciar uma sessão em MATLAB	2
	Declarações e variáveis	4
	Números e expressões aritméticas	5
	A ajuda do MATLAB	7
	Definição e manipulação de matrizes	9
	Exercícios	12
2.	Operações com matrizes	13
	Transposta de uma matriz	13
	Soma de matrizes	13
	Produto de matrizes	15
	Quociente de matrizes	16
	Operações elemento a elemento	17
	Exercícios	17
3.	Funções predefinidas	18
	Funções matemáticas elementares	18
	Matrizes elementares e manipulação de matrizes	21
	Funções matriciais	24
	Interpolação e Polinómios	26
	Análise de dados	27
	Funções Matemáticas Específicas	28
	Exercícios	29
4.	Operadores	30
	Operadores relacionais	31
	Operadores lógicos	31
	Funções para operações lógicas	33
	Operações com conjuntos	35
	Exercícios	36

2	Programar em Matlab	37
1.	Ficheiros-M	37
	Scripts	37
	Functions	39
	Exercícios	41
2.	Instruções de <i>Input</i> e <i>Output</i>	42
	Exercícios	43
3.	Instruções de controle	43
	Ciclos for	43
	Ciclos while	44
	Instruções if – elseif – else	45
	Instruções switch e case	46
	Instruções break, continue, error e return	47
4.	Funções de novo!	48
5.	Gráficos - 2D	50
	Exercícios	53

Prefácio

De entre os diversos “ambientes de computação” atualmente disponíveis, um dos mais frequentemente utilizados, para fins educacionais, é o MATLAB[®]. Este sistema apresenta características que justificam a sua ampla divulgação e utilização em diversos cursos de ciências e engenharia:

- é extremamente fácil de utilizar, sendo os dados introduzidos e manipulados de uma forma simples, especialmente no caso de vetores ou matrizes;
- inclui uma linguagem de programação de alto nível e bastante acessível, especialmente adequada a jovens que vão tomar um primeiro contacto com uma linguagem de programação;
- tem suporte em muitos sistemas computacionais diferentes, proporcionando, em grande medida, uma independência de plataforma. Programas escritos em MATLAB podem migrar para novas plataformas quando as necessidades do utilizador se alteram.
- tem um grande número de funções matemáticas predefinidas que podem ser usados no âmbito de outras disciplinas da licenciatura em matemática, praticamente desde o início do curso;
- dispõe de boas capacidades gráficas, permitindo uma visualização, de forma simples, dos dados e resultados;
- é possível adquirir uma variedade de pacotes de programas (*toolboxes*) para fins mais específicos.

Em resumo, com o MATLAB os alunos podem realizar num ambiente agradável, tarefas de natureza diversa: desenvolvimento e análise de algoritmos, modelação, computação científica, visualização, etc.

Iniciação ao MATLAB

1. Introdução

⇒ O sistema MATLAB

O MATLAB é um sistema gráfico que integra a capacidade de realização de cálculos, programação e visualização gráfica num ambiente interativo bastante agradável. Este *software* é produzido pela companhia americana



e o seu nome deriva do inglês **Matrix Laboratory**". Este sistema é particularmente popular em

1. Matemática e Computação
2. Desenvolvimento de algoritmos
3. Modelação simulação e prototipagem
4. Análise e visualização de dados
5. Desenvolvimento de aplicações

O MATLAB trabalha com matrizes quadradas ou retangulares, com elementos reais ou complexos, derivou dos projetos LINPACK e EISPACK que são considerados como a origem de algum do melhor *software* numérico disponível para computação com matrizes.

Para além da manipulação fácil de matrizes, o MATLAB permite o acesso a um número crescente de rotinas incorporadas, potencialidades gráficas a duas e três dimensões e a possibilidade de configurar o *software* às necessidades de cada utilizador.

O MATLAB possui ainda um conjunto de funções específicas - *toolboxes* - para uma dada aplicação e/ou tecnologia, como por exemplo, Estatística e Machine Learning, Otimização, Processamento de Sinal e de Imagem, Computação Paralela, etc.

O sistema MATLAB possui cinco partes principais:

1. As ferramentas para utilização e desenvolvimento;

2. A biblioteca de funções matemáticas;
3. A linguagem de programação (de alto nível);
4. As funções gráficas;
5. As bibliotecas de interfaces.

As versões atuais do MATLAB permitem realizar muitas tarefas recorrendo às suas *interfaces* gráficas - menus, botões, etc. Na maior parte dos casos o recurso a estas potencialidades pode ser feito de uma forma natural e intuitiva, pelo que os presentes textos darão mais ênfase aos comandos *escritos*.

⇒ Iniciar uma sessão em MATLAB

Quando se invoca o MATLAB¹, é criada uma janela com várias “subjanelas” e menus. Este ambiente de trabalho pode ser personalizado, de acordo com as preferências do utilizador.

Do lado direito do ecrã aparece, por defeito, a janela de comandos - *Command Window*, através da qual é possível comunicar com o interpretador do MATLAB. Quando aparece o símbolo `>>`, o MATLAB está pronto a receber instruções. A janela de comandos pode ser apagada através do comando `clc` ou usando a opção **Command Window** do botão **Clear Commands** da barra de ferramentas do menu **Home**.

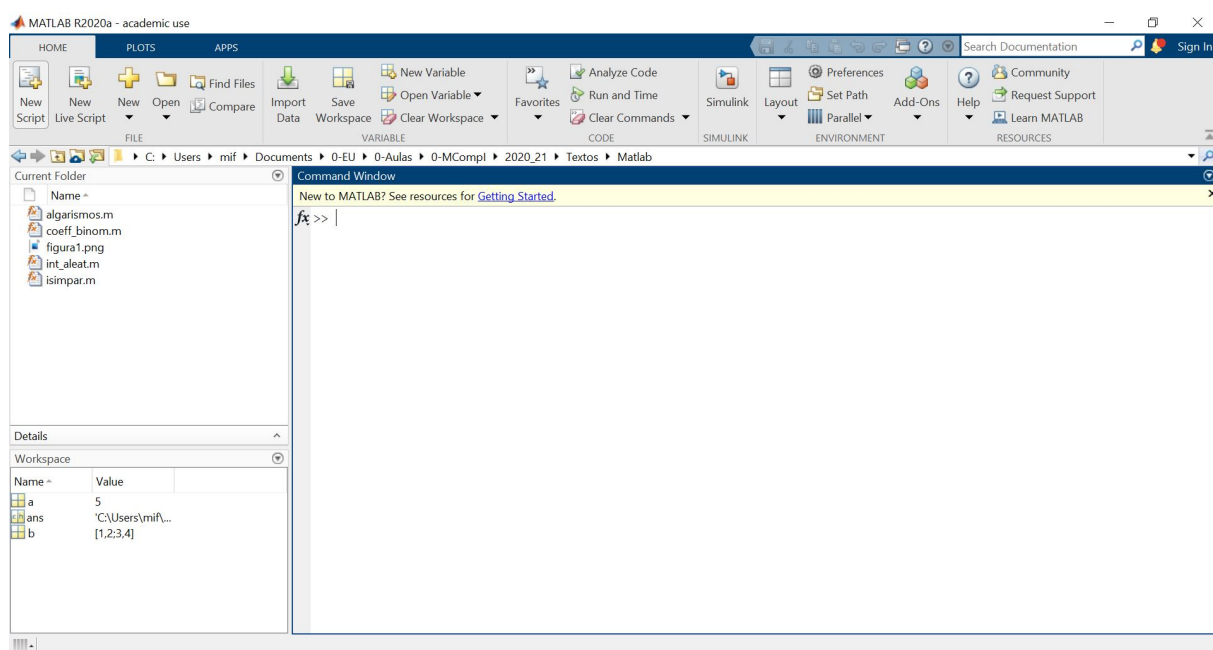


Figura 1.1: O ambiente de trabalho do MATLAB

No lado superior esquerdo pode ver-se a janela *Current Directory* onde pode ser obtida informação relativa aos ficheiros da pasta de trabalho do MATLAB. Em geral, o MATLAB só reconhece ficheiros que estejam nessa pasta ou no caminho de pesquisa do sistema *-path*. Os menus e botões desta janela podem ser usados para examinar os ficheiros, ou alternativamente, podem ser usados comandos equivalentes na janela de comandos: `pwd` - retorna o nome da pasta corrente; `cd` - altera a pasta de

¹A versão utilizada nestes textos é a versão R2020b.

trabalho; **dir** - lista todos os ficheiros da pasta corrente; **what** - lista apenas os ficheiros MATLAB. Os comandos **delete** e **type** permitem apagar um ficheiro e mostrar na janela de comandos um ficheiro, respetivamente.

```
>> pwd
```

```
ans =
```

```
'C:\Users\mif\Documents\0-EU\0-Aulas\0-MCompI\2020_21\Textos\MATLAB'
```

```
>> what
```

```
MATLAB Code files in the current folder
```

```
C:\Users\mif\Documents\0-EU\0-Aulas\0-MCompI\2020_21\Textos\MATLAB
```

```
algarismos    coeff_binom  int_aleat    isimpair
```

Todas as variáveis usadas na sessão de trabalho são gravadas no espaço de trabalho - *MATLAB Workspace*.

O conteúdo de *Workspace* pode também ser visto através do comando **whos** ou **who**.

```
>> whos
```

Name	Size	Bytes	Class	Attributes
a	1x1	8	double	
ans	1x66	132	char	
b	2x2	32	double	

```
>> who
```

```
Your variables are:
```

```
a    ans    b
```

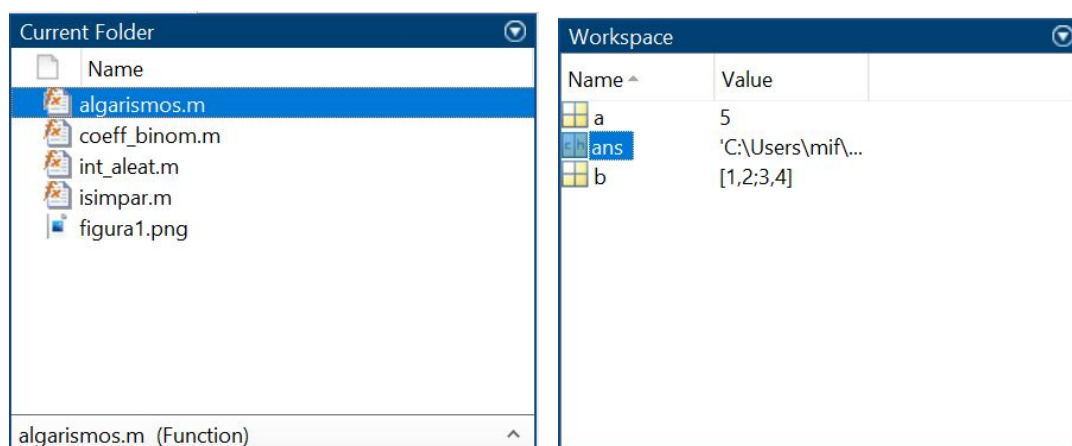


Figura 1.2: As janelas *Current Directory* e *Workspace*

As variáveis definidas ou obtidas numa sessão de trabalho podem também ser guardadas num ficheiro para serem posteriormente usadas, recorrendo aos comandos **save** e **load**. Assim, o comando

```
>> save variaveis
```

guarda todas as variáveis do espaço de trabalho num ficheiro chamado *variaveis.mat*. Para recuperar estes valores basta fazer

```
>> load variaveis
```

é ainda possível guardar apenas algumas variáveis. Por exemplo, o comando

```
>> save apenas X Y
```

guarda as variáveis X e Y num ficheiro chamado *apenas.mat*.

Para apagar a variável X, pode usar-se o comando `clear X`. Todas as variáveis do espaço de trabalho serão apagadas, se for usado apenas `clear`. Alternativamente pode usar-se o botão **Clear Workspace** da barra de ferramentas do menu **Home** ou seleccionar directamente na janela *Workspace* as variáveis a apagar e usando os botões do rato.



Figura 1.4. *Command History* do MATLAB

Finalmente, a opção **Layout** da barra de ferramentas do menu **Home** permite alterar a disposição das janelas e incluir ou retirar janelas. Em particular, na janela *Command History*, pode ver-se um historial de todas as sessões de trabalho realizadas, desde que se apagou pela última vez esta informação (opção **Command History** do botão **Clear Commands**). Verifique o que acontece se fizer um duplo clique numa das linhas do *Command History*.

⇒ declarações e variáveis

As declarações no MATLAB são frequentemente da forma

variavel=*expressão*

ou simplesmente

expressão.

No primeiro caso, o valor de *expressão* é atribuído à variável *variavel* para uso futuro. Quando o nome da variável é omitido (assim como o sinal =), o MATLAB cria automaticamente uma variável com o nome *ans* (de *answer*). Por exemplo, as declarações

```
>> a=2*3
```

```
e
```

```
>> 2*4
```

originam, respetivamente

```
a =
```

```
6
```

```
ans =
```

```
8
```

O nome de uma variável deve sempre começar por uma letra, seguido de um conjunto de letras ou números (ou ainda o sinal `_`), até ao máximo de 63 carateres. O MATLAB **distingue as letras minúsculas das maiúsculas**. Assim, a e A não representam a mesma variável. **Todas as funções do MATLAB têm nomes escritos em minúsculas** (ver Secção 1.3).

Sempre que se pretenda que o resultado de uma operação ou atribuição não apareça no ecrã (evitando assim o aparecimento de uma quantidade de informação pouco útil), deve usar-se o símbolo `;`. A declaração

```
>> a=2*3;
```

atribui o valor 6 à variável a , mas não mostra no ecrã o resultado dessa atribuição.

Quando a expressão é muito complicada, necessitando de mais de uma linha para ser definida, deve usar-se o símbolo `...` antes de mudar de linha, para indicar que a expressão continua.

```
>> 2*cos(3)+ 3*cos(2)-5*cos(1)+ 8*cos(5)-0.5*i*cos(4)+...
2*sin(3)+ 3*i*sin(2)-5*sin(1);
```

Em contrapartida, se as expressões forem muito simples, podem ser definidas simultaneamente na mesma linha de comandos, separando cada expressão através do símbolo `,`.

```
>> a=2*3,b=2*4
```

```
a =
```

```
6
```

```
b =
```

```
8
```

⇒ Números e expressões aritméticas

O MATLAB usa a notação decimal convencional para representar números, sendo também possível incluir-se, na representação de um número, potências de base 10. Assim, as expressões 0.001 e 1e-3 representam o mesmo número. A notação usada para a unidade imaginária é o i ou j . As expressões $1+2i$ e $1+2j$ representam o número complexo cuja parte real é 1 e a parte imaginária é 2.

As expressões podem ser construídas usando os operadores aritméticos usuais e correspondentes precedências. Assim, tem-se

+ adição
 - subtração
 * multiplicação
 / divisão à direita²
 \ divisão à esquerda
 ^ potenciação

O MATLAB tem incorporadas funções matemáticas elementares tais como **abs**, **sqrt**, **log**, etc. Para uma lista completa destas e outras funções, veja a Secção 1.3.

Por defeito, o MATLAB trabalha no sistema de numeração de norma IEEE em formato duplo, isto é, no sistema $F(2, 53, -1021, 1024)$ ³. Para controlar o formato de saída dos resultados pode usar-se o comando **format**. Por defeito, o MATLAB apresenta os resultados no **format short** que corresponde ao uso de 5 dígitos. É possível alterar este formato, usando formatos que permitem mais dígitos ou usam a notação científica, como se exemplifica na tabela abaixo.

Formato	Descrição
format	o mesmo que short
format short	notação de vírgula fixa, com 5 dígitos
format long	notação de vírgula fixa, com 15 dígitos
format short e	notação de vírgula flutuante, com 5 dígitos
format long e	notação de vírgula flutuante, com 15 dígitos
format short g	o melhor dos formatos de vírgula fixa ou flutuante, com 5 dígitos
format long g	o melhor dos formatos de vírgula fixa ou flutuante, com 15 dígitos

Por exemplo, a atribuição

```
>> x=sqrt(2), y=pi*1000
```

tem como resultado

```
x =
    1.4142
y =
    3.1416e+03
```

que corresponde ao formato por defeito, isto é, **short**:

```
>> format short,x,y
x =
    1.4142
y =
    3.1416e+03
```

Os exemplos seguintes ilustram outros formatos existentes.

²As operações com matrizes tornam conveniente o uso de dois símbolos para a divisão; ver Secção 1.2.

³Relembre a matéria já lecionada sobre este tema

Exemplo

```
>> format short e,x,y
x =
    1.4142e+00
y =
    3.1416e+03

>> format short g,x,y
x =
    1.4142
y =
    3141.6

>> format long,x,y
x =
1.414213562373095
y =
3.141592653589793e+03

>> format long e,x,y
x =
    1.414213562373095e+00
y =
    3.141592653589793e+03

>> format long g,x,y
x =
    1.4142135623731
y =
    3141.59265358979
```

⇒ A ajuda do MATLAB

O MATLAB disponibiliza várias formas de obter ajuda sobre uma função. A primeira delas é através do uso da função **help** cuja sintaxe é:

help nome

ou apenas

help

No primeiro caso é exibido um texto de ajuda para a funcionalidade especificada por nome, enquanto no segundo caso é mostrado conteúdo relevante para ações anteriores.

Suponhamos que pretendemos ajuda sobre a função **imag**. Para isso basta fazer

```
>> help imag
```

obtendo-se

```
imag    Complex imaginary part.
imag(X) is the imaginary part of X.
```

See I or J to enter complex numbers.

See also real, isreal, conj, angle, abs.

...

Se tiver sido feita a atribuição `a=imag(pi+2i)`, o uso de

```
>> help
```

resulta em

```
i - Imaginary unit.
imag - Complex imaginary part.
pi - 3.1415926535897....
```

Uma outra forma de obter ajuda é usando a função **doc** para aceder a uma determinada página do navegador de ajuda. Ao executar a instrução:

```
>> doc imag
```

deverá visualizar a página referente à função **imag** (ver Figura 1.5)

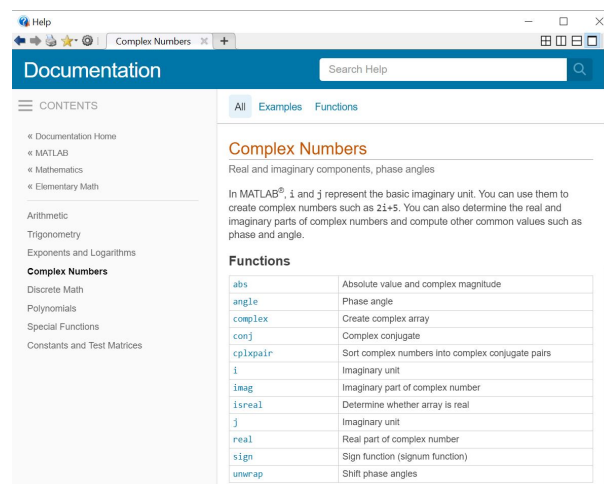


Figura 1.5. doc imag

Outra facilidade do MATLAB pode ser obtida recorrendo ao uso de **lookfor nome** que permite encontrar todas as funções que contêm a palavra *nome* na primeira linha de comentário do texto do *help*. Por exemplo,

```
>> lookfor imaginary
```

tem como resultado

```
imag - Complex imaginary part.
i - Imaginary unit.
j - Imaginary unit.
```

⇒ definição e manipulação de matrizes

O MATLAB trabalha essencialmente com um tipo de objeto, uma matriz numérica retangular com elementos eventualmente complexos. Para o MATLAB, uma matriz é um *array*⁴ bidimensional; escalares são entendidos como matrizes 1×1 ; vetores são matrizes $1 \times n$ ou $n \times 1$. Assim, as operações e comandos em MATLAB devem ser entendidos numa forma natural, enquanto operações entre matrizes.

A maneira mais simples de definir uma matriz pequena é introduzir explicitamente os seus elementos da seguinte forma: cada elemento de uma linha da matriz é separado pelo símbolo , (ou espaço) e cada linha da matriz é separada por ; (ou mudança de linha). Os elementos da matriz devem estar rodeados pelos símbolos [e] .

Por exemplo as atribuições

```
>> A=[1,1,1;2,2,2;3,3,3];
>> B=[ 1 1 1
      2 2 2
      3 3 3];
>> C=[1 1 1;2 2 2;3 3 3];
```

produzem exatamente a mesma matriz:

```
1    1    1
2    2    2
3    3    3
```

Em alguns casos, não é necessário definir explicitamente todos os elementos de uma matriz. O símbolo : permite definir vetores e/ou matrizes “especiais” de modo muito simples. Por exemplo,

```
>> x=-2:2
x =
   -2   -1    0    1    2
```

```
>> y=-2:2:6
y =
   -2    0    2    4    6
```

```
>> z=[0:3:9;2:2:8;5:5:21]
z =
    0    3    6    9
    2    4    6    8
    5   10   15   20
```

Para obter um determinado elemento de uma matriz, basta indicar a sua linha e coluna. Por exemplo, a instrução

⁴um *array* é um conjunto de dados que podem ser acedidos por indexação. No MATLAB, o conjunto de índices é sempre uma sequência de inteiros que começa em 1

```
>> z(3,2)
```

devolve o elemento da matriz z que está na linha 3 e coluna 2, isto é,

```
ans =
    10
```

Uma das vantagens do MATLAB é a de não ser necessário dimensionar as matrizes *a priori*. A cada nova instrução é feito o redimensionamento da matriz⁵. Assim, considerando a matriz A inicial, as declarações

```
>> A(5,5)=1,B(5,1)=1
```

produzem as novas matrizes

```
A =
     1     1     1     0     0
     2     2     2     0     0
     3     3     3     0     0
     0     0     0     0     0
     0     0     0     0     1
```

```
B =
     1     1     1
     2     2     2
     3     3     3
     0     0     0
     1     0     0
```

Outra forma simples de obter matrizes maiores à custa de matrizes menores já definidas é a seguinte: se pretendermos acrescentar à matriz A inicial a linha 4 4 4, basta fazer

```
>> linha=[4 4 4];A=[A;linha]
```

ou apenas

```
>> A=[A;4 4 4]
```

para se obter a nova matriz

```
A =
     1     1     1
     2     2     2
     3     3     3
     4     4     4
```

Vetores com componentes inteiras podem também ser usados como índices. Por exemplo, considerando a matriz

⁵este procedimento não é contudo aconselhável.

```
>> A=[1 2 3;4 5 6;7 8 9;-1 -2 -3;1 1 1]
```

a declaração

```
>> B=A(1:2:5,[1,3])
```

resultará numa matriz que contém as linhas 1, 3 e 5 e as colunas 1 e 3 da matriz inicial,

```
B =
     1     3
     7     9
     1     1
```

enquanto a atribuição

```
>> C=A(1:3,:)
```

resultará numa matriz que tem as 3 primeiras linhas de A .

```
C =
     1     2     3
     4     5     6
     7     8     9
```

Pode ainda usar-se **end** para referir o índice mais elevado de uma dimensão. Por exemplo,

```
>> A(end-1,end)
```

resultará no elemento da matriz A que está na penúltima linha e última coluna, isto é,

```
ans =
    -3
```

Vetores como índices podem aparecer em ambos os lados de uma atribuição. Por exemplo, sendo E a matriz

```
>> E=[1 2 3; 2 4 2;3 0 3];
```

a atribuição

```
>> A([1 3],:)=E([2 1],:)
```

resultará na matriz

```
A =
     2     4     2
     4     5     6
     1     2     3
    -1    -2    -3
     1     1     1
```

Também é possível apagar linhas e colunas de uma matriz. Para retirar a segunda linha da matriz A basta fazer

```
>> A(2,:)=[]
    2     4     2
    1     2     3
   -1    -2    -3
    1     1     1
```

⇒ Exercícios

Exercício 1. Defina, no MATLAB, a matriz

$$A = \begin{pmatrix} 1 & -1 & 2 \\ 3 & 2 & -1 \\ 1 & -5 & 4 \end{pmatrix}$$

e obtenha de uma forma simples, a partir de A , as seguintes matrizes.

$$A_1 = \begin{pmatrix} 1 & -1 & 2 \\ 3 & 2 & 1 \\ 1 & 5 & 4 \end{pmatrix} \quad A_2 = \begin{pmatrix} 1 & -1 & 2 \\ 0 & 2 & 1 \\ 0 & 0 & 4 \end{pmatrix} \quad A_3 = \begin{pmatrix} 1 & -1 & 2 & 1 \\ 3 & 2 & -1 & 2 \\ 1 & -5 & 4 & 3 \end{pmatrix}$$

$$A_4 = \begin{pmatrix} 1 & -1 & 2 \\ 3 & 2 & -1 \\ 1 & -5 & 4 \\ 1 & 2 & 3 \end{pmatrix} \quad A_5 = \begin{pmatrix} 2 & 1 & -1 & 2 \\ 0 & 1 & -1 & 2 \\ 0 & 3 & 2 & -1 \\ 0 & 1 & -5 & 4 \end{pmatrix} \quad A_6 = \begin{pmatrix} 1 & -1 & 2 & 0 & 0 \\ 3 & 2 & -1 & 0 & 0 \\ 1 & -5 & 4 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 2 \end{pmatrix}$$

$$A_7 = \begin{pmatrix} 1 & -1 & 2 & 1 & -1 & 2 \\ 3 & 2 & 1 & 3 & 2 & 1 \\ 1 & 5 & 4 & 1 & 5 & 4 \\ 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \end{pmatrix} \quad A_8 = \begin{pmatrix} 1 & -1 & 2 \\ 3 & 2 & 1 \end{pmatrix} \quad A_9 = \begin{pmatrix} 1 & -1 \\ 3 & 2 \end{pmatrix}.$$

Exercício 2. Construa o vetor $z=[10,20,30,40,50,60,70,80]$. Sem usar o MATLAB, indique que vetores se obtêm se efetuarmos cada um dos seguintes comandos.

- | | |
|---------------------------|---------------------------|
| a. >> u=z(1:2:7) | b. >> v=z(7:-2:1) |
| c. >> w=z([3 4 8 1]) | d. >> z(1:2:7)=zeros(1,4) |
| e. >> z(7:-2:1)=ones(1,4) | f. >> z(1:3)=[] |

Verifique, no MATLAB, as suas respostas.

2. Operações com matrizes

⇒ Transposta de uma matriz

O símbolo ' denota a transposta de uma matriz, no caso em que esta é real, ou a transconjugada de uma matriz, se esta for complexa.⁶

As declarações

```
>> A=[1 2 3;4 5 6;7 8 9]
>> B=A'
>> X=[-1+i 2-i 3]'
```

resultam nas matrizes

```
A =
     1     2     3
     4     5     6
     7     8     9
B =
     1     4     7
     2     5     8
     3     6     9
X =
-1.0000 - 1.0000i
 2.0000 + 1.0000i
 3.0000
```

⇒ Soma de matrizes

Os símbolos + e − designam soma e subtração de matrizes e estas operações estão definidas de forma usual. Assim, a atribuição

```
>> C=A+B
```

resulta na matriz

```
C =
     2     6    10
     6    10    14
    10    14    18
```

No MATLAB é ainda possível definir a operação $A + B$ ou $A - B$ sempre que uma das matrizes tem ordem 1, i.e. é um escalar ou é um vetor (matriz $1 \times n$ ou $n \times 1$). No primeiro caso, a matriz resultante é a que se obtém somando ou subtraindo o escalar a todos os elementos da outra matriz.

As declarações

⁶A transposta de uma matriz complexa A pode obter-se fazendo $A.'$ ou $\text{conj}(A')$.

```
>> D=A-1;
>> D=-ones(3)+A
```

originam exatamente a mesma matriz

```
D =
    0     1     2
    3     4     5
    6     7     8
```

Já no segundo caso, a operação corresponde à operação entre a matriz dada e uma matriz com todas as linhas/colunas iguais ao vetor linha/coluna e com a dimensão adequada à operação.

Exemplo

```
>> vlinha=A(1,:)
vlinha =
     1     2     3

>> repmat(linha,3,1)
ans =
     1     2     3
     1     2     3
     1     2     3

>> A+ans
ans =
     2     4     6
     5     7     9
     8    10    12

>> A+vlinha
ans =
     2     4     6
     5     7     9
     8    10    12

>> vcoluna=A(:,1)
vcoluna =
     1
     4
     7

>> repmat(vcoluna,1,3)
ans =
     1     1     1
     4     4     4
     7     7     7
```

Exemplo (cont.)

```
>> A+ans
ans =
     2     3     4
     8     9    10
    14    15    16

>> A+vcoluna
ans =
     2     3     4
     8     9    10
    14    15    16
```

⇒ Produto de matrizes

O símbolo $*$ denota multiplicação de matrizes e esta operação está definida da forma usual. Por exemplo, se

$$A = \begin{pmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{pmatrix}, \quad B = \begin{pmatrix} 1 & 2 \\ 3 & 4 \\ 5 & 6 \end{pmatrix}, \quad x = (1 \ 2 \ 3), \quad y = (4 \ 5 \ 6),$$

então as atribuições

```
>> M1=A*B
>> M2=4*A
>> M3=x'*y
```

originam as matrizes

```
M1 =
    22    28
    49    64
    76   100

M2 =
     4     8    12
    16    20    24
    28    32    36

M3 =
     4     5     6
     8    10    12
    12    15    18
```

e os comandos

```
>> M4=B*A;
>> M5=x*y
>> M6=A*x
```

produzem

```
??? Error using ==> mtimes
Inner matrix dimensions must agree.Error using *
Incorrect dimensions for matrix multiplication. Check that the number of columns
in the first matrix matches the number of rows in the second matrix...
```

uma vez que as operações indicadas não estão definidas.

⇒ Quociente de matrizes

Embora o quociente de matrizes não esteja definido, os símbolos $\backslash$ e $/$ representa a “divisão” de duas matrizes no sentido que se descreve de seguida.

Se A é uma matriz quadrada invertível, então $A \backslash B$ e B/A correspondem formalmente ao produto à esquerda e à direita, de B pela inversa de A , isto é,

$$A \backslash B = A^{-1} * B$$

$$B/A = B * A^{-1}$$

Como seria de esperar, o resultado destas operações não passa pelo cálculo explícito da inversa de A , mas sim pela resolução de sistemas⁷. Em geral

$$X = A \backslash B \text{ é solução da equação } A * X = B$$

$$Y = B/A \text{ é solução da equação } Y * A = B$$

Sejam

$$A = \begin{pmatrix} 1 & 2 & 0 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{pmatrix} \quad \text{e} \quad B = \begin{pmatrix} 3 & -3 & 1 \\ 15 & 12 & 13 \\ 24 & 18 & 19 \end{pmatrix}$$

Então, se

```
>> X=A\B,Y=A/B
```

obtem-se

X =

```
1.0000    1.0000   -1.0000
1.0000   -2.0000    1.0000
1.0000    3.0000    2.0000
```

Y =

```
-0.4103   -1.2051    0.8462
-0.0256    2.4872   -1.3846
-0.1026    2.9487   -1.5385
```

Naturalmente que se as matrizes forem de ordem 1, i.e. escalares, as duas divisões correspondem à divisão usual. Assim, $2 \backslash 8 = 8/2 = 4$ e $8 \backslash 2 = 2/8 = 0.25$.

⁷Este problema foi estudado com detalhe na unidade curricular de Álgebra Linear.

⇒ Operações elemento a elemento

As operações elemento a elemento (ou ponto a ponto) diferem das operações usuais com matrizes, mas podem ter grande aplicação prática. Para indicar que uma dada operação é para ser feita elemento a elemento deve usar-se o ponto (.) imediatamente antes do operador.

Considerando as matrizes $A = (a_{ij})$, $B = (a_{ij})$, $C = (a_{ij})$, $X = (x_{ij})$ e $Y = (y_{ij})$, as atribuições

$$A = X.^Y, \quad B = X.*Y \quad \text{e} \quad C = X./Y,$$

têm, respetivamente, o seguinte significado:

$$a_{ij} = (x_{ij})^{y_{ij}}, \quad b_{ij} = x_{ij} * y_{ij} \quad \text{e} \quad c_{ij} = x_{ij}/y_{ij}.$$

Para que estas operações possam ser executadas, as matrizes (ou vetores) X e Y têm que ter (em geral) a mesma ordem. Note-se que $X.+Y$ e $X.-Y$ não estão definidas (Porquê?).

Consideremos as matrizes

$$X = \begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix} \quad \text{e} \quad Y = \begin{pmatrix} 2 & 3 \\ 3 & 0 \end{pmatrix}$$

As seguintes operações elemento a elemento

```
>> E1=X.*Y,E2=Y./X,E3=X.^Y
```

produzem as matrizes

E1 =

```
2     6
9     0
```

E2 =

```
2.0000    1.5000
1.0000         0
```

E3 =

```
1     8
27    1
```

⇒ Exercícios

Exercício 3. Defina as matrizes

```
>> x=[1 4 2], y=[-1+i -i 1+i]
```

```
>> A=[1 2 3;-1 3 -2], B=[1 -1 1;0 1 2; 1 1 1]
```

Verifique quais das seguintes operações estão bem definidas e, em caso afirmativo, comente o resultado obtido.

a. >> x + y	b. >> x + A	c. >> x' + y'
d. >> A - [x ; y]	e. >> [x ; y']	f. >> [x ; y]
g. >> [A B]	h. >> [A ; B]	i. >> A - 3
j. >> A + B	k. >> A * B	l. >> B * A
m. >> A .* B	n. >> B * A'	

3. Funções predefinidas

O MATLAB contém um conjunto grande de funções já definidas. Algumas dessas funções são funções matriciais, como por exemplo $\text{inv}(A)$, que calcula a inversa de uma matriz quadrada A (invertível), outras estão definidas elemento a elemento. Por exemplo, $C = \cos(A)$ é uma matriz cujos elementos c_{ij} são os valores do cosseno dos elementos de $A = (a_{ij})$, i.e. $c_{ij} = \cos(a_{ij})$.

Segue-se, a título de exemplo, uma lista das funções mais usadas, dividida em 6 categorias: funções matemáticas elementares, matrizes elementares e manipulação de matrizes, funções matriciais, polinómios, análise de dados e funções matemática específicas. Algumas destas funções são ainda *estranhas* aos alunos do 1^o ano. Ao longo do ano, elas poderão vir a ser especialmente úteis e apreciadas nas várias unidades curriculares do plano de estudos do curso. Uma lista completa de todas as funções existentes em cada uma destas categorias pode ser obtida invocando o **help**. Por exemplo, `help elfun`, `help elmat`, `help matfun`, `help polyfun`, `help datafun`, `help specfun`.⁸

⇒ Funções matemáticas elementares

⇒ Funções Trigonométricas

sin	- Seno.	sec	- Secante.
sinh	- Seno hiperbólico.	sech	- Secante hiperbólica.
asin	- Arco-seno.	asec	- Arco-secante.
asinh	- Arco-seno hiperbólico.	asech	- Arco-secante hiperbólica.
cos	- Cosseno.	csc	- Cossecante.
cosh	- Cosseno hiperbólico.	csch	- Cossecante hiperbólica.
acos	- Arco-cosseno.	acsc	- Arco-cossecante.
acosh	- Arco-cosseno hiperbólico.	acsch	- Arco-cossecante hiperbólica.
tan	- Tangente.	cot	- Cotangente.
tanh	- Tangente hiperbólica.	coth	- Cotangente hiperbólica.
atan	- Arco-tangente.	acot	- Arco-cotangente.
atanh	- Arco-tangente hiperbólica.	acoth	- Arco-cotangente hiperbólica.

⁸As diversas categorias de funções encontram-se em (pasta instalação)\MATLAB\R2020b\toolbox\MATLAB.

Exemplo

```
>> sin([0 pi/2 pi 3*pi/2 2*pi])
ans =
    0    1.0000    0.0000   -1.0000   -0.0000

>> atan([0 Inf])
ans =
    0    1.5708
```

⇒ Função Exponencial

exp	- Exponencial.
log	- Logaritmo neperiano.
log10	- Logaritmo na base 10.
log2	- Logaritmo na base 2.
sqrt	- Raiz quadrada.
nthroot	- Raíz índice n de um número real.

Exemplo

```
>> exp(1)
ans =
    2.7183

>> log2(8)
ans =
    3

>> nthroot(16,4)
ans =
    2
```

⇒ Funções Complexas

abs	- Módulo.
conj	- Conjugado.
imag	- Parte imaginária.
real	- Parte real.
isreal	- Retorna verdadeiro para arrays reais.

Exemplo

```
>> z=-3+4i;
>> abs(z)
ans =
    5

>> conj(z)
ans =
   -3.0000 - 4.0000i

>> real(z)
ans =
   -3
```

⇒ Arredondamento e resto da divisão

fix	- Arredondamento na direção de zero.
floor	- Arredondamento na direção de $-\infty$.
ceil	- Arredondamento na direção de $+\infty$.
round	- Arredondamento na direção do inteiro mais próximo.
mod	- Resto da divisão, com sinal.
rem	- Resto da divisão.
sign	- Sinal.

Exemplo

```
>> x=-sqrt(7)
x =
    -2.6458

>> fix(x)
ans =
    -2

>> floor(x)
ans =
    -3

>> ceil(x)
ans =
    -2

>> round(x)
ans =
    -3

>> rem(-7,2),mod(-7,2)
ans =
    -1

ans =
     1

>> rem(-7,0), mod(-7,0)
ans =
    NaN

ans =
    -7

>> sign(x)
ans =
    -1
```

Note-se que **rem** (x, y), para $x \neq y$ e $y \neq 0$ (no MATLAB, **mod** ($x, 0$) = x), tem o mesmo sinal de x ; **mod** (x, y) e **rem** (x, y) são iguais se x e y têm o mesmo sinal, mas diferem de y se x e y têm sinais diferentes.

⇒ Matrizes elementares e manipulação de matrizes

⇒ Matrizes elementares

zeros	- Matriz nula.
ones	- Matriz com todos os elementos 1.
eye	- Matriz identidade.
repmat	- Replicar e agrupar array.
repelem	- Replicar elementos de um array.
linspace	- Vetor de elementos igualmente espaçados.

Exemplo

```
>> A1=eye(3)
A1 =
     1     0     0
     0     1     0
     0     0     1

>> A2=zeros(2,3)
A2 =
     0     0     0
     0     0     0

>> A3=2*ones(1,3)
A3 =
     2     2     2

>> A4=[1 2;3 4];

>> repmat(A4,2,3)
ans =
     1     2     1     2     1     2
     3     4     3     4     3     4
     1     2     1     2     1     2
     3     4     3     4     3     4

>> repelem(A4,2,3)
ans =
     1     1     1     2     2     2
     1     1     1     2     2     2
     3     3     3     4     4     4
     3     3     3     4     4     4

>> linspace(-1,1,5)
ans =
-1.0000 -0.5000         0     0.5000     1.0000
```

➡ Informação básica

size - *Tamanho do array.*
length - *Tamanho do vetor.*
numel - *Número de elementos.*
disp - *Exibir matriz ou texto.*
height - *Número de colunas.*
width - *Número de linhas.*

Exemplo

```

>> a= repmat([1;2],1,4)
a =
     1     1     1     1
     2     2     2     2

>> b=a(:) '
b =
     1     2     1     2     1     2     1     2

>> size(a)
ans =
     2     4

>> size(b)
ans =
     1     8

>> length(a)
ans =
     4

>> length(b)
ans =
     8

>> [numel(a);numel(b)]
ans =
     8
     8

>> [height(a) height(b);width(a) width(b)]
ans =
     2     1
     4     8

>> disp(a)
     1     1     1     1
     2     2     2     2

```

⇒ Variáveis especiais e constantes

<code>ans</code>	- Resposta mais recente.
<code>eps</code>	- Epsilon da máquina.
<code>realmax</code>	- Maior número em vírgula flutuante.
<code>realmin</code>	- Menor positivo em vírgula flutuante.
<code>pi</code>	- 3.1415926535897....
<code>i,j</code>	- Unidade imaginária.
<code>inf</code>	- Infinito.
<code>NaN</code>	- Not-a-Number.

Exemplo

```
>> eps
ans =
    2.2204e-16

>> realmax
ans =
    1.7977e+308

>> realmin
ans =
    2.2251e-308

>> 0/0
ans =
    NaN

>> 1/0
ans =
    Inf

>> pi=5 % as funções/constantes não estão protegidas
pi =
5

>> clear pi, pi % para recuperar o valor original
ans =
3.1416
```

⇒ Manipulação de matrizes

<code>reshape</code>	- Redimensionar uma matriz.
<code>diag</code>	- Criar ou extrair diagonais.
<code>tril</code>	- Extrair parte triangular inferior.
<code>triu</code>	- Extrair parte triangular superior.
<code>fliplr</code>	- Rodar a matriz na direção esquerda/direita.
<code>flip</code>	- Trocar a ordem dos elementos.
<code>rot90</code>	- Rodar uma matriz 90 graus.
<code>find</code>	- Encontrar os índices dos elementos não nulos.

Exemplo

```

>> a=reshape(1:9,3,3)
    1     4     7
    2     5     8
    3     6     9
>> diag(a)
ans =
    1
    5
    9
>> diag(ans)
ans =
    1     0     0
    0     5     0
    0     0     9
>> triu(a)
ans =
    1     4     7
    0     5     8
    0     0     9
>> [fliplr(a) flip(a)]
ans =
    7     4     1     3     6     9
    8     5     2     2     5     8
    9     6     3     1     4     7
>> rot90(a)
ans =
    7     8     9
    4     5     6
    1     2     3
>> rem(a,3)
ans =
    1     1     1
    2     2     2
    0     0     0
>> find(ans)'
ans =
    1     2     4     5     7     8

```

⇒ Funções matriciais

norm - Norma matricial ou vetorial.
rank - Característica de uma matriz.
det - Determinante de uma matriz.
trace - Traço de uma matriz.
null - Núcleo de uma matriz.

Exemplo

```

>> a=reshape(1:9,3,3);
>> rank(a)
ans =
     2

>> det(a)
ans =
     0

>> trace(a)
ans =
    15
  
```

⇒ Equações Lineares

$\backslash$ e / - Solução de um sistema linear; use "help slash".
linsolve - Solução de um sistema linear.
inv - Inversa de uma matriz.

Exemplo

```

>> a=[1 1 1;3 4 5;3 6 10];b=eye(3,1);
>> a\b
ans =
    10.0000
   -15.0000
     6.0000

>> linsolve(a,b)
ans =
    10.0000
   -15.0000
     6.0000

>> inv(a)
ans =
    10.0000    -4.0000     1.0000
   -15.0000     7.0000    -2.0000
     6.0000    -3.0000     1.0000
  
```

⇒ Valores Próprios

eig - Valores e vetores próprios.
poly - Polinómio característico.
hess - Forma de Hessenberg.

Exemplo

```
>> a=[-5 -4 0;9 5 -5;-3 0 6];
>> [vetores valores]=eig(a)
vetores =
    0.5263    0.4650    0.4082
   -0.7895   -0.8137   -0.8165
    0.3158    0.3487    0.4082

valores =
    1.0000         0         0
         0    2.0000         0
         0         0    3.0000

>> poly(a)
ans =
    1.0000   -6.0000   11.0000   -6.0000
```

⇒ Interpolação e Polinómios

As funções correspondentes à interpolação serão estudadas no 2º ano em Análise Numérica, pelo que não se listam aqui.

⇒ Polinómios

roots - Raízes de um polinómio.
poly - Construção de um polinómio, dadas as suas raízes.
polyval - Avaliar um polinómio.
polyder - Derivada de um polinómio.
conv - Produto de polinómios.
deconv - Divisão de polinómios.

Exemplo

```
>> p=poly([1,2,3,4]) % 0 polinómio p=(x-1)(x-2)(x-3)(x-4)
p =
     1    -10     35    -50     24

>> roots(p) % zeros do polinómio
ans =
    4.0000
    3.0000
    2.0000
    1.0000
```

Exemplo (cont.)

```
>> polyval(p,[1,6]) % p(1) e p(6)
ans =
    0   120

>> polyder(p) % derivada de p, i.e. -50+70x-30x^2+4x^3
ans =
    4   -30    70   -50

>> deconv(p,[1,-1]) % divisão de p por x-1, i.e. (x-2)(x-3)(x-4)
ans =
    1    -9    26   -24

>> poly([2,3,4])
ans =
    1    -9    26   -24

>> conv(ans,[1,-1]) % o produto de (x-2)(x-3)(x-4) por x-1, i.e. p
ans =
    1   -10    35   -50    24
```

⇒ **Análise de dados**⇒ **Operações básicas**

sum	- Soma dos elementos.
prod	- Produto dos elementos.
min	- Mínimo.
max	- Máximo.
mean	- Média.
median	- Mediana.
mode	- Moda.
std	- Desvio padrão.
var	- Variância.
sort	- Ordenação.

Exemplo

```
>> x=[2 -2 -1 8 -2 7 3 9 -2 2];
>> [sum(x);prod(x)]
ans =
    24
 48384

>> [min(x); max(x)]
ans =
   -2
    9
```

Exemplo (cont.)

```
>> [mean(x);median(x);mode(x)]
ans =
    2.4000
    2.0000
   -2.0000

>> [std(x) std(x)^2 var(x)]
ans =
    4.2999    18.4889    18.4889
>> sort(x)
ans =
   -2    -2    -2    -1     2     2     3     7     8     9

>> sort(x,'ascend')
ans =
   -2    -2    -2    -1     2     2     3     7     8     9

>> sort(x,'descend')
ans =
     9     8     7     3     2     2    -1    -2    -2    -2
```

⇒ Funções matemáticas específicas**⇒ Teoria de números**

factor	- Decomposição em fatores primos..
isprime	- Verdadeiro, se o argumento é um número primo.
primes	- Gerar uma lista de números primos.
gcd	- Máximo divisor comum.
lcm	- Mínimo múltiplo comum.
perms	- Permutações.
nchoosek	- Combinações.
factorial	- Fatorial.

Exemplo

```
>> factor(24) % fatorização em primos de 24=2^3*3
ans =
     2     2     2     3

>> isprime(24) % 24 não é primo
ans =
logical
     0
```

Exemplo (cont.)

```
>> primes(24) % lista de primos inferiores a 24
ans =
     2     3     5     7    11    13    17    19    23
>> gcd(24,16) % máximo divisor comum entre 24 e 16
ans =
     8
>> lcm(3,5) % mínimo múltiplo comum entre 3 e 5
ans =
    15
>> perms([1 2 3]) % todas as permutações dos elementos 1,2,3
ans =
     3     2     1
     3     1     2
     2     3     1
     2     1     3
     1     3     2
     1     2     3
>> nchoosek(6,4) % combinações dos 6, 4 a 4
ans =
    15
>> factorial(6)/(factorial(4)*factorial(2))
ans =
    15
```

⇒ Exercícios

Exercício 4. Defina, no MATLAB, o vetor linha x constituído:

- por uma sequência decrescente de números inteiros com início em 5 e término em -5 ;
- pelos números múltiplos de 3 compreendidos entre 10 e 132, dispostos por ordem decrescente;
- por 5 números aleatórios, usando a função **rand**;
- por 6 números inteiros aleatórios entre 5 e 10, usando a função **randi**;
- por 10 números igualmente espaçados do intervalo $[1, 2]$, usando a função **linspace**.
- por 128 elementos com a seguinte forma $(0 \ 1 \ 0 \ -1 \ 0 \ 1 \ \dots \ 0 \ -1)^T$.
- pelos valores da função $\sin(\pi t)$, para $t = 0 : 0.25 : 2$.

Exercício 5. Seja $x = 1.253$. Calcule os valores de y, z, w, u abaixo indicados (confronte os seus resultados com os que são indicados):

Expressão	Resultado
$y = \frac{1}{\sqrt{2\pi}} e^{-\frac{1}{2}x^2}$	0.1820
$z = x \arctan x - \frac{1}{2} \ln(1 + x^2)$	0.6523
$w = \frac{\sqrt[3]{5 + \cos 4x}}{ \sin 3x }$	3.0107
$u = \sin^2(\pi x)$	0.5094

Exercício 6. Tente antecipar o resultado de cada um dos seguintes comandos e verifique posteriormente o resultado:

- a) `>> A=reshape(1:16,4,4);` b) `>> b=sum(A)`
c) `>> c=sum(sum(A))` d) `>> D=1./A`
e) `>> l=A(3,:); E= repmat(1,3,2)` f) `>> F=diag(diag(A))`

Exercício 7. Use a função **pascal** para definir uma matriz de Pascal A de ordem 6.

- a) Verifique que a matriz A é simétrica.
b) Calcule a soma de todos os elementos da matriz A .
c) Qual é a mediana dos elementos de A ?
d) Quantos números primos tem A ? (Sugestão: use a função **find** e **isprime**.)
e) Qual é o índice de linha e coluna desses primos?

4. Operadores

A linguagem MATLAB usa muitos dos operadores que normalmente são usados para realizar operações simples em arrays de qualquer tipo. Já foram descritos anteriormente os operadores aritméticos. Daremos agora destaque aos operadores relacionais, lógicos e funções associadas, referindo também as operações entre conjuntos.

⇒ Operadores relacionais

No MATLAB, os operadores relacionais são os seguintes:

Operadores relacionais	Descrição
==	igual
<=	menor ou igual
>=	maior ou igual
~=	diferente
<	menor
>	maior

O resultado da aplicação de um operador relacional é do tipo lógico - **logical** - e pode ser **1 (true)** ou **0 (false)**. Assim,

```
>> 2>3
```

resulta em

```
logical
0
```

enquanto

```
>> 2==4-2
```

devolve

```
logical
1
```

⇒ Operadores lógicos

Adicionalmente podem ainda formar-se expressões lógicas mais complicadas combinando elementos lógicos e expressões de relação, com os seguintes operadores lógicos.

Operadores lógicos	Descrição
& (and)	e
(or)	ou
~ (not)	negação
xor	ou exclusivo
&&,	operadores de curto-circuito

Relembre as tabelas de verdade das seguintes operações:

p	~ p	p	q	p q	p	q	xor(p,q)	p	q	p&q
V	F	V	V	V	V	V	F	V	V	V
V	F	V	F	V	V	F	V	V	F	F
F	V	F	V	V	F	V	V	F	V	F
		F	F	F	F	F	F	F	F	F

Nas chamadas operações de curto-circuito, o segundo argumento é apenas executado ou avaliado se o primeiro argumento não for suficiente para determinar o valor da expressão: quando o primeiro argumento da função AND é avaliado como falso, o valor global é falso e quando o primeiro argumento da função OR for avaliado como verdadeiro, o valor global é verdadeiro.

Exemplo

```
>> ~(2>3,2==4-2)
ans =
    1×2 logical array
     1     0

>> (2<3)|(2>3)
ans =
    logical
     1

>> (2<3)&(2>3)
ans =
    logical
     0
```

O resultado da aplicação de um operador relacional a matrizes da mesma dimensão é uma matriz com 0's e 1's resultante da aplicação do operador elemento a elemento.

Exemplo

```
>> A=[1 2;3 4]; B=[2 2;1 5];
>> A>=2
ans =
    2×2 logical array
     0     1
     1     1

>> A>B
ans =
    2×2 logical array
     0     0
     1     0
```

Arrays lógicos podem também ser usados como índices. Obtém-se neste caso um array com os valores para os quais o índice é 1. Por exemplo,

```
>> a=[1 2 3 4];
>> a>2
ans =
    1×4 logical array
     0     0     1     1

>> a(ans)
```

```
ans =
     3     4
```

Note-se que as instruções

```
>> b=[0 0 1 1];
>> a(b)
```

resultam em

Array indices must be positive integers or logical values.

porque b é um *array* de reais. Teria que ser convertido num *array* lógico para que a operação fosse possível (veja secção seguinte).

```
>> a(logical(b))
ans =
     3     4
```

⇒ Funções para operações lógicas

Para além dos já referidos operadores lógicos, o MATLAB fornece um conjunto de outras funções para realizar operações lógicas. A tabela seguinte lista algumas dessas funções.

all	- Determinar se todos os elementos de um array são não nulos ou true.
any	- Determinar se algum dos elementos de um array é não nulo ou true.
false	- Falso (logical 0).
true	- Verdadeiro (logical 1).
find	- Determinar índices e valores de elementos não nulos.
islogical	- Determinar se o argumento é do tipo logical.
logical	- Converter valores numéricos em lógicos.

Exemplo

```
>> a=randi([-5,9],[3,4])
a =
    -4     6    -4     7
     9     7     0     1
    -5     8    -2     8

>> all(a>0)
ans =
    1×4 logical array
     0     1     0     1
```

Exemplo (cont.)

```

>> all(a>0,'all')
ans =
logical
0

>> any(a<0,2)
ans =
    3×1 logical array
     1
     0
     1

>> find(a>7)
ans =
     2
     6
    12

>> a(ans)
ans =
     9
     8
     8

>> a(a>7)
ans =
     9
     8
     8

>> [linha coluna]=find(a>7)
linha =
     2
     3
     3

coluna =
     1
     2
     4

```

Para além da função **islogical** e da função já mencionada **isprime**, existem ainda várias outras funções cujo nome começa por **is** e cujo resultado é **true/false**.

A título de exemplo referem-se as funções **isdiag**, **isempty**, **isletter**, **isnan**, **istril**, cuja designação é suficientemente sugestiva para antecipar o seu objetivo (confirme através do help e de exemplos).

⇒ Operações com conjuntos

As operações com conjunto comparam os elementos de dois conjuntos para encontrar semelhanças ou diferenças entre eles. As operações mais frequentes estão listadas de seguida.

intersect	- <i>Interseção.</i>
ismember	- <i>Determinar elementos que pertencem a um conjunto.</i>
setdiff	- <i>Diferença entre dois arrays.</i>
union	- <i>União.</i>
unique	- <i>Valores únicos de um array.</i>

Exemplo

```
>> a=[-3 -2 -2 -1 -3]; b=[-1 4 -1 -3];
>> intersect(a,b)
ans =
    -3    -1

>> union(a,b)
ans =
    -3    -2    -1     4

>> unique(a)
ans =
    -3    -2    -1

>> union(a,a)
ans =
    -3    -2    -1

>> setdiff(a,b)
ans =
    -2

>> setdiff(b,a)
ans =
     4

>> ismember(a,b)
ans =
    1×5 logical array
     1     0     0     1     1

>> ismember(b,a)
ans =
    1×4 logical array
     1     0     1     1
```

Exemplo (cont.)

```
>> a=reshape(1:9,3,3);
>> b=repelem(a(1:2,1:2),2,1);
>> intersect(a,b)'
ans =
     1     2     4     5

>> setdiff(a,b)'
ans =
     3     6     7     8     9

>> ismember(b,a)
ns =
4x2 logical array
     1     1
     1     1
     1     1
     1     1
```

Para obter uma lista das funções predefinidas existentes nesta categoria, faça `help ops`.

⇒ Exercícios

Exercício 8. Dado $x = [1 \ 5 \ 2 \ 8 \ 9 \ 0 \ 1]$ e $y = [5 \ 2 \ 2 \ 6 \ 0 \ 0 \ 2]$ execute e explique o resultado dos seguintes comandos:

- | | |
|--|--|
| a. <code>>> x > y</code> | b. <code>>> y < x</code> |
| c. <code>>> x == y</code> | d. <code>>> x <= y</code> |
| e. <code>>> y >= x</code> | f. <code>>> x y</code> |
| g. <code>>> x & y</code> | h. <code>>> x & (~y)</code> |
| i. <code>>> (x > y) (y < x)</code> | j. <code>>> (x > y) & (y < x)</code> |

Exercício 9. Dado $x = 1:10$ e $y = [3 \ 1 \ 5 \ 6 \ 8 \ 2 \ 9 \ 4 \ 7 \ 0]$ execute e interprete o resultado dos seguintes comandos:

- | | |
|---|--|
| a. <code>>> x(x > 5)</code> | b. <code>>> y(x <= 4)</code> |
| c. <code>>> x((x < 2) (x >= 8))</code> | d. <code>>> y((x < 2) (x >= 8))</code> |
| e. <code>>> y((x > 2) & (x < 8))</code> | f. <code>>> x(y < 0)</code> |

Exercício 10. Defina o vetor $x=[3 \ 15 \ 9 \ 12 \ -1 \ 0 \ -12 \ 9 \ 6 \ 1]$ e indique o(s) comando(s) para:

- atribuir o valor zero aos elementos de x que são positivos;
- multiplicar os elementos pares de x por 5.

Programar em Matlab

1. Ficheiros-M

As instruções em MATLAB são geralmente dadas e executadas linha a linha. É, no entanto, possível executar uma sequência de comandos que está guardada num ficheiro. Ficheiros que contêm código em linguagem MATLAB são chamados *ficheiros-M* (em inglês, *M-files*) e são do tipo **nome_ficheiro.m**.

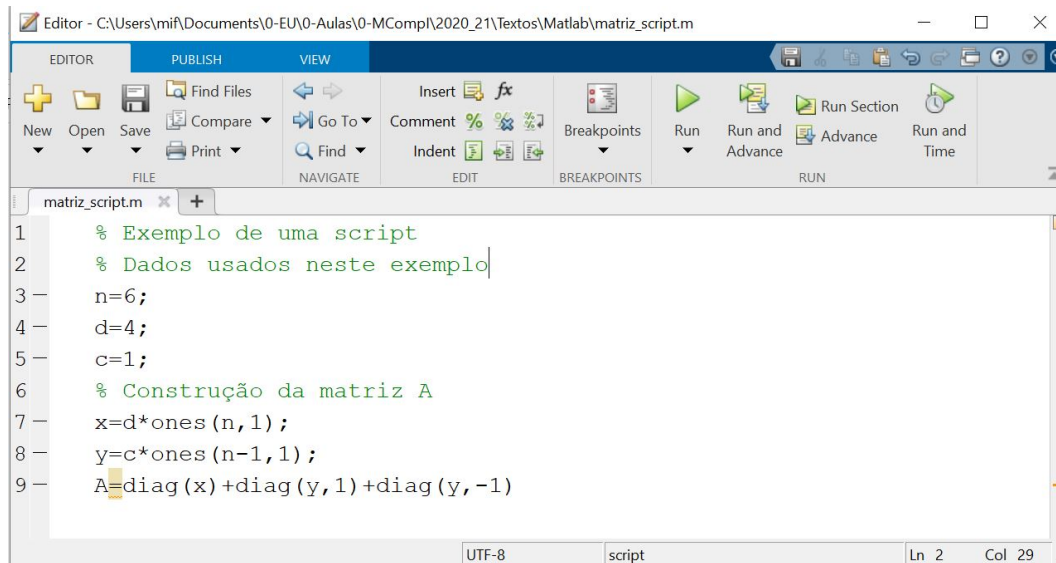
Um ficheiro-M consiste numa sequência de comandos MATLAB que pode inclusivamente fazer referência a outros ficheiros-M. Os ficheiros-M que vamos usar podem ser de dois tipos: *script* ou *function*.

⇒ *Scripts*

Para criar uma *script* basta seleccionar, na barra de ferramentas, a opção **New** seguida de **Script** ou usar em alternativa a opção **New Script** da mesma barra de ferramentas; para editar uma *script* já existente deve usar-se a opção **Open**.

Quando uma *script* é invocada, o MATLAB executa os comandos encontrados no ficheiro e as variáveis que aparecem na *script* podem ser de novo usadas. Este tipo de ficheiro é muito útil quando há necessidade de executar um grande número de operações.

Suponhamos, por exemplo, que se pretende gerar uma matriz tridiagonal de ordem 6×6 que tem o valor 4 ao longo da diagonal principal e o valor 1 nas diagonais abaixo e acima da diagonal principal. Para o efeito, na barra de ferramentas do MATLAB, seleccione a opção **New Script**, para criar um ficheiro de texto chamado, por exemplo, *matriz_script.m* com a seguinte sequência de instruções:



Para executar esta *script* basta seleccionar o botão **Run** do menu *Editor* ou escrever, na janela do MATLAB

```
>> matriz_script
```

obtendo-se

A =

4	1	0	0	0	0
1	4	1	0	0	0
0	1	4	1	0	0
0	0	1	4	1	0
0	0	0	1	4	1
0	0	0	0	1	4

A matriz A e os vetores x e y estão disponíveis e podem ser de novo utilizados; por exemplo, se fizer

```
>> sum(x)-sum(y)
```

obterá

ans =

19

Naturalmente que se quisermos alterar os dados deste problema, terá que ser editado o ficheiro *matriz_script.m* para alteração dos valores. Uma alternativa mais elegante e eficaz de alterar os valores dos parâmetros sem ter que alterar o código envolve as chamadas *functions* que a seguir se descrevem.

É ainda possível dividir uma *script* em várias secções que podem ser executadas separadamente. Para isso deve usar-se `%%` para separar as secções e o botão **Run Section** para executar cada secção. Qual o valor de x_1 e x_2 se executar a segunda secção da seguinte script?

```

1  %% Primeira Secção
2  n=6;d=4;c=1;
3  x=d*ones(n,1);
4  y=c*ones(n-1,1);
5  A=diag(x)+diag(y,1)+diag(y,-1)
6  %% Segunda Secção
7  a=1;b=2;c=3;
8  D=sqrt(b^2-4*a*c);
9  x1=(-b+D)/(2*a);
10 x2=(-b-D)/(2*a);

```

⇒ Functions

Um ficheiro-M que contém a palavra **function** no início da primeira linha é chamado uma função (*function*). As funções diferem das *scripts* na medida em que podem ser usados argumentos e as variáveis definidas e manipuladas dentro de uma função são locais, i.e. não operam globalmente no espaço de trabalho.

Uma *function* é definida do seguinte modo:

function *parâmetros_saida*=*nome_função*(*parâmetros_entrada*)

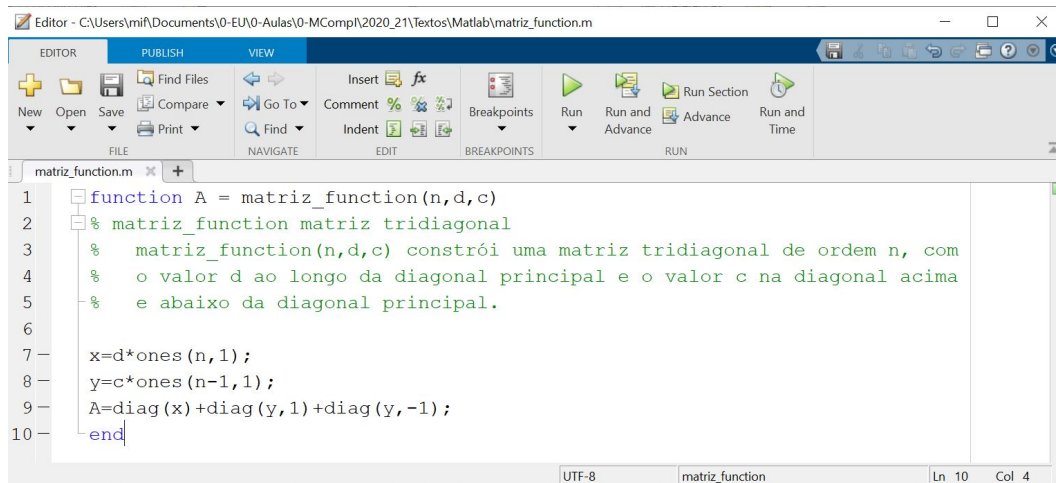
Quando se selecciona a opção **Function** do menu **New**, é criado automaticamente um ficheiro com o seguinte aspeto.

```

1  function [outputArg1,outputArg2] = untitled4(inputArg1,inputArg2)
2  %UNTITLED4 Summary of this function goes here
3  % Detailed explanation goes here
4  outputArg1 = inputArg1;
5  outputArg2 = inputArg2;
6  end

```

Para resolver o problema anteriormente considerado da construção de uma matriz tridiagonal, pode criar-se a função *matriz_function.m*, alterando convenientemente o ficheiro anterior (o ficheiro deve ser **sempre** gravado com o nome “nome_função” atrás referido.)



A função `matriz_function` passa a fazer parte das funções do MATLAB. As primeiras linhas de comentário que aparecem na função podem ser acedidas via *help*.

```

>> help matriz_function
matriz_function matriz tridiagonal
matriz_function(n,d,c) constrói uma matriz tridiagonal de ordem n, com
o valor d ao longo da diagonal principal e o valor c na diagonal acima
e abaixo da diagonal principal.

```

Para construir a matriz A definida anteriormente basta fazer

```
>> matriz_function(6,4,1)
```

```
ans =
```

```

4     1     0     0     0     0
1     4     1     0     0     0
0     1     4     1     0     0
0     0     1     4     1     0
0     0     0     1     4     1
0     0     0     0     1     4

```

Neste caso as variáveis x e y não estão definidas no espaço de trabalho.

```
>> x
```

```
Unrecognized function or variable 'x'.
```

```
>> y
```

```
Unrecognized function or variable 'x'.
```

O caso de funções com mais de um parâmetro de entrada ou saída é tratado de modo análogo. A função seguinte calcula as raízes de um polinómio do segundo grau.

```

function [x1,x2]=raizes(a,b,c)
% raizes Calcula as raízes da equação ax^2+bx+c=0,
D=sqrt(b*b-4*a*c);
x1=(-b+D)/(2*a);
x2=(-b-D)/(2*a);

```

Listagem 1. Função com dois parâmetros

Para resolver, por exemplo, a equação $x^2 + x + 2 = 0$ basta fazer

```
>> [r1,r2]=raizes(1,1,2)
```

obtendo-se, então:

```
r1 =  
-0.5000 + 1.3229i
```

```
r2 =  
-0.5000 - 1.3229i
```

⇒ Exercícios

Exercício 11. Considere as seguintes funções MATLAB

```
function y=f1(x)  
perimetro=2*pi*r;  
area=pi*r^2;  
end
```

```
function area=f3(r)  
perimetro=2*pi*r;  
area=pi*r^2;  
end
```

```
function [perimetro,area]=f5(r)  
perimetro=2*pi*r;  
area=pi*r^2;  
end
```

```
function y=f2(r)  
perimetro=2*pi*r  
area=pi*r^2  
end
```

```
function [area,perimetro]=f4(r)  
perimetro=2*pi*r;  
area=pi*r^2;  
end
```

```
function [area,perimetro]=f6(r)  
perimetro=2*pi*r;  
area=pi*r.^2;  
end
```

- a) se $r = 1$, qual o resultado das seguintes instruções (use o MATLAB apenas para verificar as suas respostas)?

```
>> f1(r)    >> area=f3(1)    >> [x,y]=f3(1)  
>> f2(r)    >> area=f4(1)    >> [x,y]=f4(1)  
>> f3(r)    >> area=f5(1)    >> [area,perimetro]=f5(1)
```

- b) se $r = [1, 2]$, qual o resultado das seguintes instruções?

```
>> [area,perimetro]=f5(r)  
>> [area,perimetro]=f6(r)
```

2. Instruções de *Input* e *Output*

É possível introduzir um texto ou um valor numérico através do teclado, de uma forma interativa. Para tal, pode usar-se a função **input** cuja forma é:

variavel_numerica=input('texto que vai aparecer')

ou

variavel_string=input('texto que vai aparecer','s').

No primeiro caso, aparece no ecrã o texto *texto que vai aparecer* e o utilizador deve introduzir um valor numérico que será atribuído à variável *variavel_numerica*. O segundo caso é usado quando se pretende introduzir um valor não numérico.

Como exemplo da utilização desta função, considerem-se as instruções

```
>> vn=input ('introduza o valor de vn')
```

e

```
>> vs=input ('Pretende continuar? (SIM/NÃO)', 's')
```

Se for introduzido o valor 10 no primeiro caso e SIM no segundo, então as instruções anteriores originam *vn=10* e *vs= SIM*. Para criar uma *prompt* de várias linhas, use '\n' para indicar cada nova linha; por exemplo:

```
>>vs=input ('Pretende continuar?\n Responder SIM ou NÃO\n', 's')
```

A função **disp** permite escrever um texto ou valor no ecrã. Assim, o comando

```
>> vs=disp(A)
```

escreve a matriz *A* no ecrã, enquanto que

```
>> disp('Estou a escrever esta frase' )
```

escreverá o texto *Estou a escrever esta frase*.

Para combinar texto e valores numéricos devem converter-se estes últimos a "texto" através da função **num2str**. O comando

```
>> disp(['Estou a escrever esta frase', ' há ', num2str(n), ' minutos'] )
```

escreverá no ecrã, no caso *n = 10*, a frase

```
Estou a escrever esta frase  há 10 minutos
```

Para outros formatos de entrada/saída, invoque o **help** do MATLAB para conhecer melhor o uso das instruções **fprintf**, **sprintf**, **fwrite**, **fscanf**, etc.

⇒ Exercícios

Exercício 12. Execute e comente a seguinte *script*.

```
a) graus=input('Graus? ');
   rads=graus/180*pi;
   disp([num2str(graus),' graus, corresponde a ',num2str(rads), ' radianos'])

b) x=input('x? ');
   y(x>0)=1;
   y(x<=0)=-1;
   disp(y)

c) x=1:5;
   y=sqrt(x);
   fprintf('%3i',x)
   fprintf('%3i\n',x)
   fprintf('%12.8f\n',x)
   fprintf('%12.8f\n',y)
   fprintf('%3i %12.8f\n',[x;y]);
   fprintf(' x      sqrt(x) \n'),fprintf(' %3i %12.8f\n',[x;y]);
   fprintf(' %3i %6.2e\n',[x;10*y]);
```

3. Instruções de controle

Como já foi referido, normalmente as instruções em MATLAB são executadas sequencialmente, isto é, depois de uma instrução ter sido executada, é executada a instrução imediatamente a seguir. As seguintes transferências de controle podem ser usadas para alterar a sequência de execução das instruções de uma *script* ou função: ciclos **for** e **while**, instruções **if** e **switch** e ainda **break**, **error** e **return**. Estas instruções são semelhantes às encontradas na maioria das linguagens de programação.

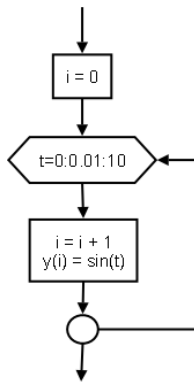
⇒ Ciclos for

A instrução **for** permite que um grupo de instruções seja executado repetidas vezes. Tem a forma

```
for variavel=expressão
    instruções
end
```

onde *expressão* é usualmente da forma **m:n** ou **m:i:n**, sendo **m** o valor inicial, **i** o valor incremental e **n** o valor final da *variavel*.

Suponhamos que pretendemos calcular um vetor com o valor da função seno em 1001 pontos igualmente espaçados do intervalo [0, 10]. A maneira mais óbvia de obter este vetor é:



```
i=0
for t=0:.01:10
    i=i+1;
    y(i)=sin(t);
end
```

Listagem 2. Ciclos for

Nota: O tempo de execução de programas em MATLAB pode ser substancialmente reduzido, se se tiver a preocupação de “vetorizar” os algoritmos. Vetorizar significa converter ciclos em operações com vetores ou matrizes. Por exemplo as instruções,

```
>> t=0:.1:10;
>> y=sin(t);
```

correspondem a uma versão vetorizada deste exemplo.

Note-se que `t=0:.01:10` é apenas um vetor linha. De facto, qualquer vetor linha pode ser usado num ciclo **for**. Por exemplo, o seguinte conjunto de instruções

```
x=1:100;soma=0;
for j=find(isprime(x))
    soma=soma+x(j);
end
```

Listagem 3. Ciclos for - outro exemplo

calcula a soma de todos os números primos inferiores a 100.

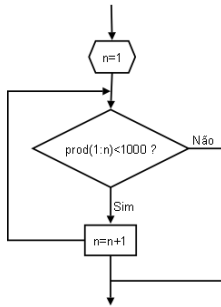
⇒ Ciclos **while**

A instrução **while** permite que um grupo de instruções seja executado um número indefinido de vezes, enquanto uma condição for satisfeita. Tem a forma

```
while expressão
    instruções
end
```

onde *expressão* é uma expressão de relação da forma $e1Re2$, sendo $e1$ e $e2$ expressões aritméticas e R um dos operadores de relação já definidos na Secção 1.4.

O exemplo seguinte ilustra o uso de um ciclo **while**, onde se pretende calcular o primeiro inteiro n para o qual $n!$ é um número com 4 dígitos.



```

n=1;
while prod(1:n)< 1000
    n=n+1;
end
n
  
```

Listagem 4. Ciclos while

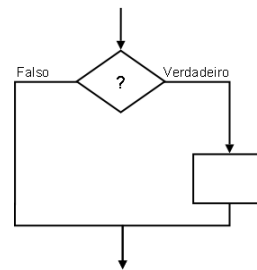
⇒ Instruções if – elseif – else

Os blocos if permitem alterar a ordem de execução de uma sequência de instruções, se determinada condição for (ou não) satisfeita.

⇒ if ... end

```

if expressão_logica
    instruções
    avaliadas, se expressão_logica verdadeira
end
  
```



```

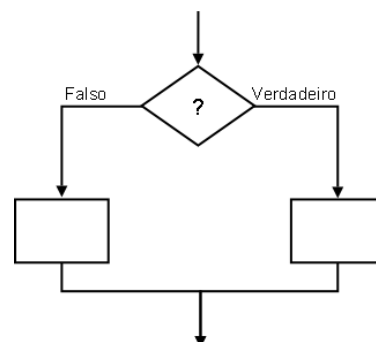
if a>0
    b=a;
    disp('a é positivo');
end
  
```

Listagem 5. Uso de if – end

⇒ if ... else ... end

```

if expressão_logica
    instruções
    avaliadas, se expressão_logica verdadeira
else
    instruções
    avaliadas, se expressão_logica falsa
end
  
```



```

if T>37
    febre=1;
    disp('Com febre');
else
    febre=0;
    disp('Sem febre');
end

```

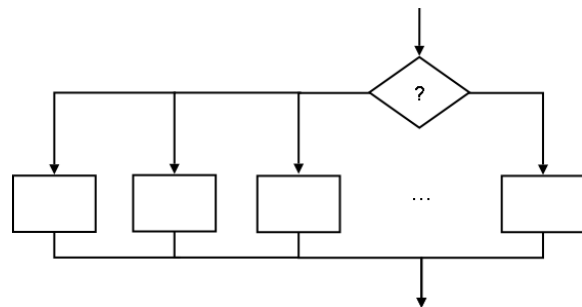
Listagem 6. Uso de if – else – end

⇒ **if ... elseif ... else ... end**

```

if expressão_logica1
    instruções
    avaliadas, se expressão_logica1 verdadeira
elseif expressão_logica2
    instruções
    avaliadas, se expressão_logica2 verdadeira
elseif expressão_logica3
    instruções
    avaliadas, se expressão_logica3 verdadeira
...
else
    instruções
    avaliadas, se nenhuma das expressões an-
    teriores é verdadeira
end

```



Cada uma das expressões *expressao_logica1*, *expressao_logica2*, etc é uma expressão de relação da forma *e1Re2*, sendo *e1* e *e2* expressões aritméticas e *R* um dos operadores de relação definidos anteriormente. Podem também combinar-se estes operadores de relação com os operadores lógicos ou ainda usar-se funções lógicas.

```

if altura>190
    disp('Muito alto');
elseif altura>170
    disp('Alto');
elseif altura<150
    disp('Baixo');
else
    disp('Mediano');
end

```

Listagem 7. Uso de if – elseif – else – end

⇒ Instruções switch e case

A instrução **switch** executa um grupo de instruções, dependendo do valor de uma variável ou expressão. Tem a forma

```

switch expressao
    case caso1
        instruções
    case {caso2a,caso2b, ...}
        instruções
    ...
    otherwise
        instruções
end

```

Se $n = 23$, qual será o resultado da seguinte *script*?

```

switch rem(n,4)
case 0
    a=ones(n)
case {1,2}
    a=eye(n)
otherwise
    a=zeros(n)
end

```

Listagem 8. Uso de `switch` – `case` – `otherwise`

⇒ Instruções `break`, `continue`, `error` e `return`

Além das instruções de controle já definidas, o MATLAB dispõe de quatro comandos - **`break`**, **`continue`**, **`error`** e **`return`**, para controlar a execução de *scripts* e funções. Uma breve descrição destas instruções é feita de seguida.

A instrução **`break`** permite sair de um ciclo **`for`** ou **`while`**. A instrução **`continue`** pode ser usada em ciclos **`for`** ou **`while`**. Quando o MATLAB encontra a instrução **`continue`** dentro de um ciclo, passa imediatamente para a instrução final desse ciclo, ignorando todas as instruções entre **`continue`** e a declaração **`end`**.

Um exemplo trivial da utilização destas instruções encontra-se na *script* apresentada na Listagem 9. Esta *script* escreve no ecrã os números entre 5 e 10.

```

for i=1:20
    if i<5
        continue
    elseif i>10
        break
    end
    disp(i);
end

```

Listagem 9. Uso de `break` e `continue`

O comando

`error('mensagem_de_erro')`

dentro de uma função ou *script*, aborta a execução, exibe a mensagem de erro *mensagem_de_erro* e faz regressar o controle ao teclado.

O comando **return** faz regressar o controle à função invocadora ou ao teclado. A Listagem 10 ilustra o uso das instruções **error** e **return**. Neste caso, pretende-se apenas obter os zeros reais de um polinómio do segundo grau, cujos coeficientes devem ser indicados num vetor.

```
function [x1,x2]=zeros_p(p)
if length(p) ~= 3
    error('p deve ser um polinómio de grau 2');
end
delta=p(2)*p(2)-4*p(1)*p(3);
if delta<0
    disp('Este polinómio não tem zeros reais');
    return
else
    x1=(-p(2)+sqrt(delta))/(2*p(1));
    x2=(-p(2)-sqrt(delta))/(2*p(1));
end
disp('Este polinómio tem zeros reais');
```

Listagem 10. Uso de **error** e **return**

4. Funções de novo!

⇒ Funções anónimas

É possível definir funções, de forma simples, através de uma linha de comando, sem recorrer à criação (e consequente armazenamento em disco) de um ficheiro-M. Esta alternativa passa pela utilização das chamadas *funções anónimas*.

A sintaxe para criar uma função anónima é a seguinte:

$$f_anonima = @(argumentos) expressão$$

Um exemplo muito simples de uma função anónima é:

```
>>g=@(x) x^2+2*x+3
```

Para avaliar esta função basta fazer

```
>> g(2)
ans =
    11
```

Nota: É boa prática, definir sempre uma função anónima de forma que possa operar sobre escalares, vetores ou matrizes. Se

```
>> f=@(x) x.^2+2*x+3
```

qual será o valor de $g(1:5)$ e $f(1:5)$?

As funções anónimas podem também ter mais que um argumento. Por exemplo, a função $h(x, y) = (x^2 + y^2, x + y)$ poderia ser definida e avaliada em $x=2$ e $y=4$ da seguinte forma

```
>> h=@(x,y) [x.^2+y.^2 x+y]
>> h(2,4)
```

Outra vantagem do uso de funções anónimas diz respeito à possibilidade de usar implicitamente variáveis definidas no espaço de trabalho, sem ter que as declarar como globais. Por exemplo:

```
>> peso1=0.4;peso2=0.5;peso3=0.1;
>> notafinal=@(T1,T2,TPC) peso1*T1+peso2*T2+peso3*TPC;
>> notafinal(8,11,14)
ans =
    10.1000
```

⇒ Quando uma função é um argumento

Muitas funções em MATLAB têm como argumentos outras funções. O MATLAB chama genericamente a estas funções **function functions** (faça `help funfun` para obter uma lista completa deste tipo de funções). Existem várias formas de passar uma função como argumento, dependendo da forma como esta foi escrita.

Uma forma é usar funções anónimas. Por exemplo, a função `fzero` permite encontrar um zero de uma função de uma variável. Podemos encontrar um zero de $f(x) = \sin(x)$ próximo de 3, fazendo

```
>> f=@(x) sin(x);
>> fzero(f,3)
ans =
    3.1416
```

é também possível passar a função `sin` como argumento, usando a sintaxe especial

```
>> fzero(@sin,3)
ans =
    3.1416
```

O nome `@sin` é chamado *function_handle* e é uma forma de referir abstratamente uma função, em vez de a invocar diretamente. *Function_handle* é um tipo de dado do MATLAB que contém toda a informação necessária para avaliar uma função. Uma função tipo *function_handle* pode também ser criada colocando o carácter **@** atrás do nome da função definida anteriormente através de um ficheiro-M.

Para obter informação sobre uma *function_handles* podem usar-se as funções **fun2str** ou **functions**.

```
>> func2str(f)
ans =
```

```
'@(x)sin(x)'
```

```
>> functions(g)
ans =
    struct with fields:

        function: '@(x)x^2+2*x+3'
           type: 'anonymous'
          file: ''
    workspace: {[1x1 struct]}
    within_file_path: ''
```

Do ponto de vista da eficiência e versatilidade, o uso de funções do tipo *function_handles* deve ser encorajado, especialmente se estas forem passadas como argumento para outras funções.

5. Gráficos - 2D

O MATLAB dispõe de um grande número de facilidades gráficas; centraremos a nossa atenção em gráficos básicos 2-D. Todos os pormenores relativos às ferramentas de visualização incluídas nesta versão do MATLAB podem ser obtidos na [página](#) da Mathworks ou invocando diretamente o **help** para `graph2d`, `graph3d`, `specgraph` ou `graphics`.

O comando mais simples e, talvez o mais útil para produzir gráficos 2-D tem a forma

`plot(Abcissas, Ordenadas, 'estilo')`

onde *Abcissas* e *Ordenadas* são vetores (com a mesma dimensão) que contêm as abcissas e ordenadas de pontos do gráfico e *estilo* é um argumento opcional que especifica o estilo da linha ou ponto a desenhar. Na tabela seguinte estão indicados alguns dos estilos que é possível definir.

cor		ponto		linha	
y	amarela	.	ponto(.)	—	contínua
m	magenta	o	círculo (o)	:	ponteadada
c	cião	x	cruz(x)	-.	'traço-ponto'
r	vermelha	+	mais(+)	--	tracejada
g	verde	*	estrela(*)		
b	azul	s	quadrado		
w	branca	d	losango(◇)		
k	preta	v	triângulo (▽)		

A função **plot** também permite o uso de apenas um vetor como argumento. Se $\text{Pontos} = [x_1 \ x_2 \ \cdots \ x_n]$, o comando

plot(Pontos)

desenha os pontos de coordenadas (i, x_i) , $i = 1, \dots, n$.

Títulos, designação dos eixos, legendas e outras características, podem ser acrescentadas a um dado gráfico, usando as funções **title**, **xlabel**, **ylabel**, **grid**, **text**, **legend**, etc. Estas funções têm a forma seguinte.

Designação	Descrição
title('título')	produz um <i>título</i> na parte superior do gráfico
xlabel('nome_x')	o eixo dos <i>xx</i> é designado por <i>nome_x</i>
ylabel('nome_y')	o eixo dos <i>yy</i> é designado por <i>nome_y</i>
grid	coloca uma quadrícula no gráfico
text(x,y,'texto_em_x_y')	escreve o texto <i>texto_em_x_y</i> na posição (x, y)
gtext('texto')	permite colocar <i>texto</i> numa posição a indicar com o rato
legend('texto1','texto2')	produz uma legenda com <i>texto1</i> e <i>texto2</i>
legend off	retira legenda

É também possível controlar os limites dos eixos através do comando **axis**. A função **axis** tem várias opções que permitem personalizar os limites, a escala, a orientação, etc, de um gráfico. Escrevendo

$$\text{axis}([xmin \ xmax \ ymin \ ymax])$$

os limites passam a ser *xmin* e *xmax* para o eixo dos *xx* e *ymin* e *ymax* para o eixo dos *yy*. Este comando deve aparecer depois do comando **plot**.

A função **axis** também aceita palavras chave para controlar os eixos. Por exemplo, **axis square**, **axis equal**, **axis auto**, **axis on**, etc. (Faça **help axis**).

Numa mesma figura podem sobrepor-se vários gráficos, recorrendo ao comando **hold**. As instruções

```
hold on
plot(x1, y1, 'estilo 1')
plot(x2, y2, 'estilo 2')
plot(x3, y3, 'estilo 3')
hold off
```

originam a sobreposição de três gráficos. Este objetivo também pode ser atingido, usando

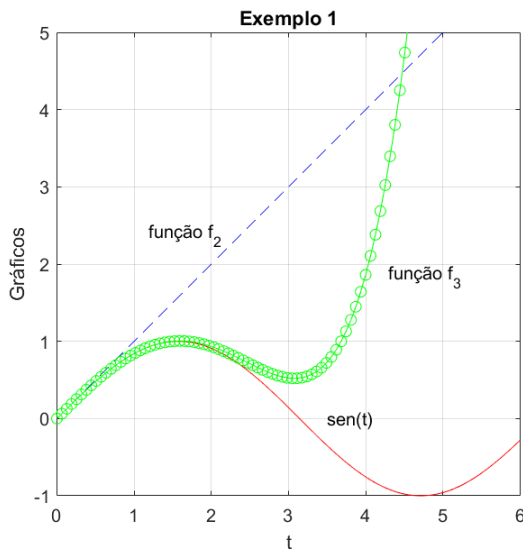
$$\text{plot}(x_1, y_1, \text{'estilo 1'}, x_2, y_2, \text{'estilo 2'}, x_3, y_3, \text{'estilo 3'})$$

O comando **hold** é especialmente útil quando o conjunto de pontos a desenhar não está disponível todo ao mesmo tempo.

Na figura seguinte estão representados simultaneamente os gráficos das funções

$$f_1(t) = \sin t, \quad f_2(t) = t, \quad f_3(t) = t - \frac{t^3}{3!} + \frac{t^5}{5!}.$$

Estes gráficos foram obtidos recorrendo à *script* apresentada na Listagem 13, onde foram usadas várias opções da instrução **plot**.



```
t=linspace(0,2*pi);
y1=sin(t);
y2=t;
y3=t-(t.^3)/6+(t.^5)/120;
plot(t,y1,'r',t,y2,'b--',t,y3,'go-')
axis equal
axis([0 6 -1 5])
grid
xlabel('t'), ylabel('Gráficos')
title('Exemplo 1')
text(3.5,0,'sen(t)')
gtext('função f_2'),gtext('função f_3')
```

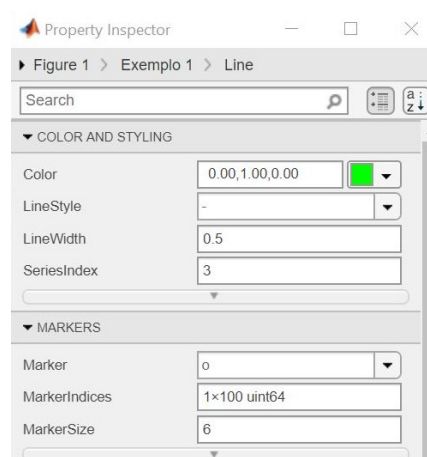
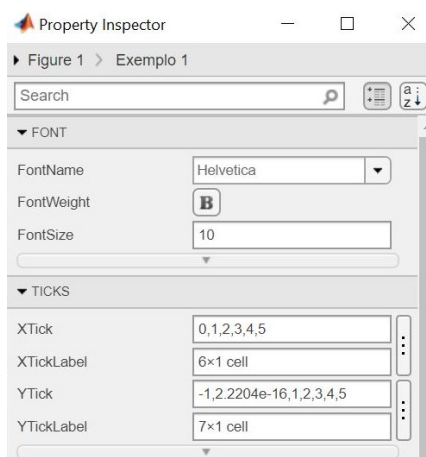
Listagem 13. A função plot

Alternativamente poder-se-ia obter o gráfico anterior, recorrendo à função **fplot**. Na listagem seguinte apresenta-se essa solução.

```
hold on
fplot(@(t)sin(t),[0,2*pi],'r')
fplot(@(t)t,[0,2*pi],'b--')
fplot(@(t) t-(t.^3)/6+(t.^5)/120,[0,2*pi],'go-')
.
.
.
hold off
```

Listagem 14. A função fplot

O MATLAB contém uma *interface* gráfica que permite criar e editar gráficos sem usar código. Para isso na janela da figura, selecione o botão *Open Property Inspector* ; irá aparecer uma janela com o aspeto da figura abaixo (lado esquerdo), onde poderá fazer as alterações. Selecionando um elemento do gráfico que se pretende alterar, aparecerá o menu correspondente. Por exemplo selecionando a linha correspondente à função f_3 irá ter acesso a um menu do tipo ilustrado no lado direito da figura.



⇒ Exercícios

Exercício 13. Considere a função $f(x) = \sin(2\pi x)$.

- Use a função **linspace** para obter uma tabela de valores da função f , em 100 pontos igualmente espaçados do intervalo $[0, 1]$.
- Use o comando **plot** para esboçar o gráfico da função no intervalo $[0, 1]$.
- Repita a alínea anterior, definindo uma função anónima e usando o comando **fplot**.

Exercício 14. Considere a função $f(x) = x \log(x) - 1$, $x > 0$, a qual se sabe ter um único zero.

- Construa uma função anónima correspondente a f .
- Por meio de um gráfico adequado, sugira um intervalo de amplitude igual a 1 que contenha o zero de f .
- Confirme a sua resposta à alínea anterior por um processo analítico.

Exercício 15. A área de um polígono regular com n lados, inscrito num círculo de raio r , é dada por

$$A_{n,r} = \frac{n}{2} \sin\left(\frac{2\pi}{n}\right) r^2.$$

- Escreva uma função anónima que retorne o valor da área de um polígono regular com n lados, inscrito num círculo de raio r .
- Faça uma tabela que ilustre o comportamento de $A_{n,1}$, quando $n \rightarrow +\infty$. Esta tabela deverá ter um aspeto análogo ao seguinte.

n	A(n,1)
1e+001	2.93892626
1e+002	3.13952598
.	.
.	.
.	.